

Final Project

Luis Gilbuena, Matthew Merritt, Michael Merritt, Anton Petrenko

May 7, 2025

1 Approach

1.1 Primary Data Structure

The key idea to our approach was to construct a dictionary that would use n -grams (represented as tuples) as keys and return the most likely part of speech for the word given that specific n -gram. This was done by going through the dataset and constructing n -grams from each sentence using the actual part of speech, and tallying the results as we went. Once all of the dataset was parsed and added to the dictionary, the dictionary was then separated into two sets:

- **“Unanimous”** n -grams that always resulted in the same part of speech during training regardless of additional context, such as a bigram that always resulted in the word being tagged as a verb.
- **“Non-unanimous”** n -grams that could result in different parts of speech with additional context, such a trigram that could result in being tagged as a noun or a verb based on the words before it.

Once these two sets are separated, we then store only the most likely option for the part of speech for every n -gram to return when queried, and remove all of the additional tallying information.

1.2 n -gram Lookup Order

Using the two sets mentioned above, we modified our approach to use the smallest amount of information needed to get a definitive answer. Specifically, we first check the unanimous set to see if the unigram always leads to the same part of speech regardless of what comes next, followed by the same check for the bigram, trigram, and quadgram. Assuming that there was no single answer beforehand, we then use the non-unanimous set to find the most likely result from the quadgram, trigram, bigram, or unigram, checking them in that order. If there are still no results for the given n -gram in either set, we default to assuming that the word is a noun (“NN”), the most common part of speech.

1.3 Jackknife Training

For the training process, we used a jackknife technique to split the data into separate training and testing datasets, with each slice created by the jackknife procedure setting aside 1000 sentences for the testing dataset, and using the remaining sentences as the training dataset. Since there were 47959 sentences in total, this means that 48 different training and testing sets were created and tested (the final testing set was only 959 sentences rather than the full 1000, since the dataset did not split evenly). The accuracy was then taken as the average of all 48 different trials.

1.4 Separate Implementations

To test out a few different implementation ideas we had as a group, we created three separate implementations of the aforementioned approach.

- **Anton’s implementation**, which uses Python’s built-in CSV module and strings as keys for the dictionaries instead of tuples.
- **Luis’s implementation**, which uses a JSON file to load the sentence data and uses sets to handle tiebreakers.

- **The Merritt’s implementation**, which uses pandas DataFrames to load the data all at once and then applies functions to columns and rows of the data to build our model.

Each implementation will be discussed more in depth in its own section below.

2 Implementations

This section focuses on the Python implementations of the general approach described above.

2.1 Anton’s Implementation

2.1.1 Implementation

In Anton’s implementation, there is only one dictionary for all the n -grams. When building the model, the unigram model is built first. The reason for this is to be able to identify the unanimous words from the non-unanimous words. Following the creation of the unigram model, another pass is done through the data. In this pass the bigram, trigram, and quadgram models are created. For efficiency, the second pass includes a lookup to the existing unigram model for each word. If the word only ever results in one part of speech, there is no reason to save more data about this word. If the word appears as multiple parts of speech, we construct each gram by inserting it into the existing dictionary with the history encoded in the key. The resulting dictionary contains all n -gram versions. Keys in the dictionary are strings of the relevant words, where any relevant history is encoded before the word using a circumflex as a delimiter.

```

1 for sentence in lines:
2     history = []
3     for word, pos in sentence:
4         if len(unigram[word].keys) > 1:
5             for i in range(len(history)):
6                 unigram[f"{'^'.join(history[-(i+1):])}^{word}"][pos] += 1
7             history.append(pos)
8         if len(history) == n:
9             history.pop(0)

```

This is a small pseudocode snippet denoting the process of building the n -grams in the second pass of the data. As shown, the unigram model is always checked before constructing the other n -grams in the same dictionary. Keys into the dictionary are strings which lead to another dictionary which contains the part of speech as keys and the number of times it appeared as that part of speech as the values. Note the creation of keys which have the history encoded in them, as well as the clearing of the history for each sentence. The array which holds the history is also bound by the highest number n denoting the highest n -gram model to create.

2.1.2 Results

The results of this implementation of the n -gram tagger were as follows:

- **Unigram only:** 95.28% accuracy, runtime of 17.720s
- **Up to bigram:** 98.07% accuracy, runtime of 26.394s
- **Up to trigram:** 98.60% accuracy, runtime of 37.253s
- **Up to quadgram:** 98.93% accuracy, runtime of 48.513s

2.2 Luis’s Implementation

2.2.1 Implementation

This approach uses the JSON library to convert the given CSV file into a JSON file where each sentence is represented as a list of (word, part-of-speech) tuples. These generated sentences are then split into two types of sets of different sizes using the jackknife method. The testing set, containing at most 1000 sentences with each sentence having their list of words associated with their true part of speech, and the training set, containing the rest of the sentences in the same format as the testing set. This

implementation proves to be more consistent compared to others which could be attributed to the model generation stage.

In the model generation portion of the code, 4 dictionaries were created corresponding to the n-grams that were to going to be later tested upon, unigram, bigram, trigram, and quadgram. Each observed unigram, bigram, trigram, and quadgram paired with their part of speech was added to a frequency table for their respective dictionary. In the case where key value pairs had a length of size 1, the key value pairs were shifted from the original 4 dictionaries and put into another list of dictionaries assigned for non-unanimous n-grams. For unianimous n-grams, the key value pairs would stay in their respected dictionaries and would later be sectioned as unanimous frequency tables/dictionaries.

Handling cases where keys would have multiple parts of speech with the equal highest frequencies was where this implementation differs. In cases where a word has multiple parts of speech with equal highest frequencies—say the word ‘accused’ has a unigram frequency dictionary of {‘VBN’:3, ‘VBD’:3}—we assign the word a set containing parts of speech tied for the highest frequency. In the case we reach the non-unanimous dictionaries, instead of checking if the true part of speech is equal to the value, given the key is in the dictionary, we check if the true part of speech is inside the set.

```

1 for x in range(4):
2     mdl = model[x]
3     for key in list(mdl.keys()):
4         if len(mdl[key]) > 1:
5             maxvalue = max(mdl[key].values())
6             maxval = set()
7             for y in mdl[key]:
8                 if mdl[key][y] == maxvalue:
9                     maxval.add(y)
10            multimodel[x][key] = maxval
11            del mdl[key]
12        else:
13            pos = [x for x in model[x][key]]
14            model[x][key] = pos[0]
15    return model + multimodel

```

After the training process, the testing model contains a list of 8 dictionaries in the following order:

```

1 [(unanimous unigram), (unanimous bigram), (unianimous trigram), (unianimous quadgram
), (non-unanimous quadgram), (non-unanimous trigram), (non-unanimous bigram), (non-
unanimous unigram)].

```

The testing process would iterate over this list of dictionaries fomr left to right and check to see if its (word, true part of speech) pair was found in the frequency tables. There are two things that can happen for each dictionary. The key would fail over to its next frequency table or the key-value pair would pass breaking the iteration and moving on to the next word. In the case were the key was not found with its correct part of speech, the model would default its guess as a noun.

2.2.2 Results

The results of this implementation of the n -gram tagger were as follows:

- **Unigram only:** 94.45% accuracy, runtime of 1m 45.964s
- **Up to bigram:** 96.67% accuracy, runtime of 1m 46.061s
- **Up to trigram:** 96.86% accuracy, runtime of 1m 45.628s
- **Up to quadgram:** 97.22% accuracy, runtime of 1m 45.106s

2.3 The Merritt’s Implementation

2.3.1 Implementation

This approach utilizes the pandas Python library for data processing, reading in data as a tabular format and applying operations to entire columns at a time to achieve faster transformations. The implementation and can be divided into four parts: initial data processing, model generation, testing against a model, and jackknife training. Each part is implemented in a separate function to allow for flexibility in terms of n -gram size, excluding initial data processing which only occurs once and does not take any arguments.

In the initial data processing stage, the tagged sentence data is read from the spreadsheet, with each row containing a single word in a sentence and the expected part of speech, and with numbers marking the beginning of sentences. To simplify the process of dividing the dataset into sentences for training and testing, the sentence numbers are cascaded down to the subsequent words that occur before the next sentence starts. Additionally, the sentence number is extracted from the original format by stripping away non-numeric characters. Finally, string data types needed to be specified and missing value detection needed to be disabled when loading the dataset in using pandas to avoid problems with empty fields or words reserved for values in Python, i.e. None and False.

In the model generation stage, two major steps occur: mapping and flattening. Before mapping can occur, the history of previous parts of speech must be generated for each word and stored in the row for later use. To do this, the part of speech column is copied and shifted down based on the number of history entries required. For example, when generating quadgrams, the history requires the previous three parts of speech, meaning that three shifted columns need to be added. Values in the shifted columns are left blank if the sentence number changes after a shift. With the history added to each row in the table, a mapping can be generated by applying a function to the rows of the table, creating a dictionary that maps history patterns to occurrence counters for each part of speech. This step of applying the function to the rows to build the dictionary is the most time consuming step in our implementation, taking roughly 15 seconds per jackknife iteration. With the mapping dictionary created, a flattening step is performed to create the model where the mappings are separated into separate dictionaries for unanimous and non-unanimous n-grams that skip past the counts for each part of speech and instead return only the most probable part of speech.

One modification explored but ultimately not implemented at this model generation stage was aimed at reducing ambiguity, but it was not included due to negative impact on accuracy. Specifically, in the case of the non-unanimous model, we explored removing cases where the part of speech decision was ambiguous. For example, when using the first thousand sentences as the testing set and the remaining sentences as the training set, one such ambiguity occurs for the the word 'ruling' after a preposition, proper noun, then possessive ending. In this case, there is an equal chance of 'ruling' representing a 'Verb, gerund or present participle' ('VBG') or a 'Noun, singular or mass' ('NN'), with both occurring 5 times in the training set. Removing this pattern from our model did not cause a noticeable increase in accuracy, so ambiguity is instead currently decided on a seemingly arbitrary basis. Currently, in the case of ties, the pattern that was observed first will be selected by the max function call on the dictionary of counts.

```

1  def generate_ngram_model(training_set, gram_size=4):
2      # ----- STAGE 0: PREPROCESSING HISTORY -----
3      modified = pd.DataFrame(training_set)
4
5      for n in range(gram_size - 1):
6          modified[f'Previous POS {n + 1}'] = modified['POS'].shift(n + 1)
7          modified[f'Previous POS {n + 1}'] = modified[f'Previous POS {n + 1}'].where(
            modified['Sentence #'] == modified['Sentence #'].shift(n + 1))
8
9      # ----- STAGE 1: INITIAL COUNTING -----
10     ngram_map = {}
11
12     def write_to_map(row):
13         previous_pos = list(map(lambda n : row[f'Previous POS {n}'], list(range(1,
14             gram_size))[::-1]))
15         previous_pos = list(filter(lambda pos : not pd.isna(pos), previous_pos))
16
17         patterns = [( *previous_pos[-i:], row['Word']) for i in range(1, len(
18             previous_pos) + 1)]
19         patterns.append(row['Word'])
20
21         for pattern in patterns:
22             if pattern in ngram_map:
23                 if row['POS'] in ngram_map[pattern]:
24                     ngram_map[pattern][row['POS']] += 1
25                 else:
26                     ngram_map[pattern][row['POS']] = 1
27             else:
28                 ngram_map[pattern] = { row['POS']: 1 }
29
30     modified.apply(write_to_map, axis=1)
31
32     # ----- STAGE 2: FLATTENING -----

```

```

31
32     # index 0 is the unanimous map, index 1 is the highest probability map
33     ngram_model = [{}, {}]
34     for (key, pos_map) in ngram_map.items():
35         map_index = 0 if len(pos_map) == 1 else 1
36
37         ngram_model[map_index][key] = max(pos_map, key=pos_map.get)
38
39     return (ngram_map, ngram_model)

```

In the testing against a model stage, the model from the previous stage is taken alongside the testing set, and each row is tested using both the unanimous and non-unanimous models when necessary. First the testing set is augmented with history similar to the training set, and then a function is applied on each row that tests and stores the outcomes into a new column. The testing order is to go through all the unanimous models from least amount of context (unigram) to most amount of context (quadgram in our case), then go through all the non-unanimous models in the opposite order. Output is appended to a text file for debugging and logging. This log file should be deleted between separate iterations of the program to prevent confusion.

In the final jackknifing stage, the raw dataset is divided into training and testing sets that can be passed to the model generation and testing stages. Each iteration selects 1000 sentences for testing and passes the remaining sentences as the training set. Between iterations, the entire model is discarded and rebuilt, with only the correct and incorrect result counts remaining across iterations. Overall accuracy is output at the end.

2.3.2 Results

Ultimately, this approach yielded lower than expected accuracy and generally performed slower than other approaches. In terms of the performance, the biggest impact was in the model generation stage when the mapping function is applied to the rows of the training set. We were able to iterate on this, having initially iterated over the rows individually, taking 2-3 minutes per training set. In the end, we reduced this down to about 15 seconds by applying a function to the rows of the table instead, but this still adds up. Regarding the lower accuracy, comparing our quadgram results to the provided sample output (99.00% accuracy), we found that only additional incorrect tests we had were when ambiguous decisions occurred, making up the 4% difference in accuracy.

The results of this implementation of the n -gram tagger were as follows:

- **Unigram only:** 93.60% accuracy, runtime of 5m 13.980s
- **Up to bigram:** 95.84% accuracy, runtime of 7m 46.116s
- **Up to trigram:** 95.94% accuracy, runtime of 10m 46.101s
- **Up to quadgram:** 95.91% accuracy, runtime of 13m 39.980s

3 Reflection

Out of the three different implementations of the base approach, Anton's implementation was the fastest and most accurate. This speed was likely the result of having less overhead by using the built-in Python CSV reading module and storing keys as strings instead of tuples, which take additional time to construct. Luis's approach performed slightly slower than Anton's approach and had slightly lower accuracy, but ran for a consistently short amount of time for all sizes of n -gram model. The Merritt's approach performed the slowest by a notable margin, having the most overhead when it came to processing the original data and building the initial dictionary to count occurrences. This was likely due to additional work required by the pandas module when augmenting the data with sentence numbers and history.

	<i>n</i> -gram Size	Accuracy	Runtime (s)
Anton	Unigram	95.28%	17.720
	Bigram	98.07%	26.394
	Trigram	98.60%	37.253
	Quadgram	98.93%	48.513
Luis	Unigram	94.45%	105.964
	Bigram	96.67%	106.061
	Trigram	96.86%	105.628
	Quadgram	97.22%	105.106
Merritt	Unigram	93.60%	313.980
	Bigram	95.84%	466.116
	Trigram	95.94%	646.101
	Quadgram	95.91%	819.980